

Herald



Protocol Research: Agent2Agent Protocol (A2A)

Field	Value
Revision	1
Created	2026-05-22
Last modified	2026-05-22
Status	research-only (no implementation yet)
Status summary	A2A (Google → Linux Foundation, contributed 2025-06; v1.0 released 2026-03-12) is the canonical agent-to-agent interop protocol with 150+ org backing (Microsoft, AWS, Salesforce, SAP, ServiceNow, Workday, IBM). Complementary to MCP — A2A models opaque agents-as-services with tasks/artifacts; MCP models tools-as-functions. IBM ACP was MERGED into A2A on 2025-08-27 (see ACP.md). Herald should adopt A2A as the SECOND non-REST protocol after MCP.
Issues	open-questions: gRPC vs JSON-RPC vs REST default; AgentCard discovery scope; signed-AgentCards adoption.
Continuation	Wave 4b — open HRD-230..HRD-250 block; new module <code>commons_a2a/</code> (L1); every flavor publishes an AgentCard.

Constitutional anchors

- **§107 anti-bluff** — every A2A test below MUST produce physical proof (real a2a CLI tool connecting to real Herald AgentCard, JSON-RPC message/send byte-level round-trip assertion).
- **§11.4.74 catalogue-check** — performed; no existing `vasic-digital` or `HelixDevelopment` A2A module. Verdict: `vendor.github.com/a2aproject/a2a-go/v2` as submodule.
- **§11.4.61 / branding** — tracked doc; regenerate HTML/PDF/DOCX siblings.
- **Spec V3 implication** — Wave 4b will add a §4x.x to `specification.V3.md`. Per §106, the spec edit triggers comprehensive implementation in the same wave.

Table of contents

- [§1. Protocol overview](#)

- [§2. Specification deep-dive](#)
- [§3. Herald-specific applicability analysis](#)
- [§4. Step-by-step implementation guide for Herald](#)
- [§5. §107 anti-bluff testing strategy](#)
- [§6. Open questions for operator](#)
- [§7. References](#)
- [§8. Catalogue-check verdict](#)

§1. Protocol overview

A2A — Agent2Agent Protocol. Open standard for agent-to-agent communication. Originally published by Google in April 2025; contributed to the Linux Foundation in June 2025; v1.0 production-ready release on 2026-03-12. Apache 2.0 license. 150+ organizations support it (Google, Microsoft, AWS, Salesforce, SAP, ServiceNow, Workday, IBM, ...) as of April 2026.

Differs from MCP how? MCP is "LLM [?] tool" (resource/tool/prompt vocabulary); A2A is "agent [?] agent" (task/artifact vocabulary). A2A treats both ends as **opaque agents** — neither needs to know the other's internals; they exchange tasks (units of work) and artifacts (results). MCP, by contrast, treats the server as a transparent tool provider. The two protocols are explicitly designed to coexist; the spec notes "ACP intentionally reuses MCP message types where compatible and doesn't prevent agents from exporting Google Agent Cards for A2A participation."

Wire format. JSON-RPC 2.0 over HTTP(S) is the primary surface. Alternative transport bindings: gRPC (binary protobuf), REST (more "OpenAPI-friendly"). All three are supported by the official Go SDK.

Transports.

- **HTTP + JSON-RPC 2.0** — default. POST to the agent's endpoint URL.
- **HTTP + SSE** — Server-Sent Events for streaming task updates (the `message/stream` method).
- **HTTP + Push Notifications** — async webhook callback for long-running tasks.
- **gRPC** — binary protobuf binding for performance-sensitive deployments.
- **REST** — RESTful endpoint binding for ecosystems that don't speak JSON-RPC.

Authentication. "Parity with OpenAPI's authentication schemes at launch":

- API keys (X-API-Key header)
- HTTP Basic / Bearer
- OAuth 2.0 / OpenID Connect (RFC 6749)
- Mutual TLS (mTLS — client certs)
- v1.0 added **Signed Agent Cards** — the AgentCard JSON itself is JWS-signed by the agent's identity provider, letting consumers verify the AgentCard's authenticity without a TLS chain assumption.

Primary use cases.

1. Multi-agent orchestration — agent A delegates a task to agent B.
2. Cross-vendor agent interop — Google's Vertex AI Agent calls into Microsoft's Copilot agent.
3. Agent marketplace — agents discover each other via AgentCards in `/well-known/agent.json`.
4. Asynchronous workflows — long-running tasks with status tracking.

Adoption. Linux Foundation governance. 150+ org members. v1.0 production deployments at Microsoft, AWS, Salesforce, SAP, ServiceNow as of April 2026. The Google Agent Development Kit (ADK) ships A2A as first-class.

§2. Specification deep-dive

§2.1 Roles

- **Client Agent** — initiates communication; consumes another agent's AgentCard + sends tasks.
- **Remote Agent (Server)** — publishes an AgentCard + serves tasks.

There is no host/server/client triangle like MCP. Agents are peers.

§2.2 AgentCard — capability advertisement

An AgentCard is a JSON document, typically served at `https://agent.example.com/.well-known/agent.json`. It declares:

- `name` — human-readable name.
- `description` — what the agent does.
- `url` — the agent's endpoint URL.
- `version` — semver of the agent.
- `capabilities` — `streaming`, `pushNotifications`, etc.
- `defaultInputModes` / `defaultOutputModes` — supported MIME types (`text/plain`, `application/json`, `audio/wav`, etc.).
- `skills` — array of skill objects, each with `id`, `name`, `description`, `tags`, `examples`, `inputModes`, `outputModes`.
- `authentication` — supported auth schemes (OAuth 2.0 / API key / mTLS / Signed AgentCards).

Example (truncated):

```
{
  "name": "Herald Project Herald (pherald)",
  "description": "Herald's project-event notification agent.
    Ingests events + fans them out.",
  "url": "https://pherald.example.com/a2a",
  "version": "0.4.0",
  "capabilities": {
    "streaming": true,
    "pushNotifications": true
  },
  "defaultInputModes": ["application/json", "text/plain"],
  "defaultOutputModes": ["application/json"],
  "skills": [
    {
      "id": "notify",
      "name": "Notify subscribers",
      "description": "Dispatch a message to tenant subscribers
        via configured channels.",
```

```

    "tags": ["notification", "fanout"],
    "examples": ["Notify the on-call team about a P1
        incident"],
    "inputModes": ["application/json"],
    "outputModes": ["application/json"]
  }
],
"authentication": {
  "schemes": ["Bearer", "OAuth2"]
}
}

```

§2.3 Tasks + artifacts

The core unit of work is a **task**:

- `id` — UUID.
- `sessionId` — optional session ID grouping related tasks.
- `status` — one of `submitted`, `working`, `input-required`, `completed`, `canceled`, `failed`, `rejected`.
- `history` — array of `Message` objects (chronological).
- `artifacts` — array of `Artifact` objects (the task's outputs).
- `metadata` — free-form key/value.

A **Message** has:

- `role` — user or agent.
- `parts` — array of `Part` objects (text, file, data — multi-modal).

A **Part** is one of:

- `TextPart { type: "text", text: "..."}`
- `FilePart { type: "file", file: { ... } }`
- `DataPart { type: "data", data: { ... } }` — structured JSON.

§2.4 Methods (JSON-RPC)

- `message/send` — send a message to an agent (creates a new task or appends to an existing one).
- `message/stream` — like `message/send` but server returns SSE stream of incremental updates.
- `tasks/get` — fetch task status + artifacts by ID.
- `tasks/cancel` — request cancellation.
- `tasks/pushNotificationConfig/set` — set the webhook URL for async completion.
- `tasks/pushNotificationConfig/get`
- `tasks/resubscribe` — re-attach to an existing task's SSE stream after disconnect.

Example `message/send`:

```

{
  "jsonrpc": "2.0",

```

```

{id": "req-1",
"method": "message/send",
"params": {
  "message": {
    "role": "user",
    "parts": [
      { "type": "text", "text": "Notify on-call about P1
incident #4321" }
    ]
  },
  "configuration": {
    "blocking": false,
    "acceptedOutputModes": ["application/json"]
  }
}
}

```

§2.5 Lifecycle

1. **Discovery.** Client fetches `https://agent.example.com/.well-known/agent.json`.
2. **Optionally verify.** If `signedAgentCard`, verify JWS.
3. **Authenticate.** Per the authentication block.
4. **Send.** `message/send` or `message/stream`.
5. **Poll / stream.** `tasks/get` (sync) or SSE stream (`message/stream`) or push webhook.
6. **Complete or cancel.** Task terminal state.

§2.6 Streaming + push semantics

- **Sync.** `message/send` with `configuration.blocking: true` returns the completed task in one round-trip (for short tasks).
- **Streaming (SSE).** `message/stream` returns a `text/event-stream` of incremental `TaskStatusUpdateEvent` and `TaskArtifactUpdateEvent` events.
- **Push.** Long-running task — client preregisters a webhook via `tasks/pushNotificationConfig/set`; the agent POSTs back when done.

§2.7 Versioning

A2A version is in the AgentCard's `version` field (semver). The PROTOCOL version is implicit per spec date. v1.0 is current; the protocol has a backwards-compat policy similar to MCP.

§2.8 Signed AgentCards (v1.0 feature)

AgentCard JSON is wrapped in a JWS (JSON Web Signature) signed by the agent's issuer identity. The consumer verifies the signature against the issuer's JWKS endpoint. This eliminates the "is this AgentCard authentic?" trust problem when AgentCards are shared via untrusted channels (registries, links).

§3. Herald-specific applicability analysis

§3.1 Which flavor binaries publish A2A AgentCards?

Every **Herald flavor** publishes an AgentCard. Each flavor is itself an "opaque agent" from A2A's perspective: it has skills (notify, query, etc.) and accepts task delegations.

- **pherald** — skills: `notify`, `bulk_notify`, `subscribe`, `query_events`.
- **cherald** — skills: `query_compliance`, `attest`, `evidence_collect`.
- **sherald** — skills: `query_safety`, `acknowledge_safety_event`.
- **bherald** — skills: `query_budget`.
- **rherald** — skills: `query_risk`, `flag_risk`.
- **iherald** — skills: `acknowledge_incident`, `resolve_incident`, `query_incident`.
- **scherald** — skills: `enqueue_job`, `cancel_job`, `query_schedule`.

Each AgentCard at `https://<flavor>.example.com/.well-known/agent.json`.

§3.2 A2A vs MCP — coexistence

Both protocols are adopted. They serve different consumers:

- **MCP** — LLM clients (Claude Desktop, Cursor) — Herald is a "tool provider."
- **A2A** — agent frameworks (Google ADK, BeeAI, LangGraph) — Herald is a "peer agent."

The SAME backend (subscribers table, events_processed, audit_log, dispatchers) serves both. The translation layer in `commons_a2a/` maps A2A's `message/send` task to the same internal "dispatch_notification" call that MCP's `tools/call_notify` maps to. ONE business logic; TWO surfaces.

§3.3 Does A2A replace REST?

No — it **adds** another transport surface. REST remains the primary HTTP API; A2A is the agent-interop surface.

§3.4 Multi-tenant story

A2A AgentCards have no native tenant concept. Options:

1. **One AgentCard per tenant** — `https://pherald.example.com/tenants/{tenant_id}/.well-known/agent.json`. Each tenant's AgentCard has its own URL prefix. Simple but explodes URL count.
2. **One AgentCard, tenant in JWT** — single AgentCard at the public well-known path; `tenant_id` extracted from the OAuth 2.0 Bearer token. Preferred — matches Herald's existing JWT-tenant model.
3. **Sub-agents per tenant** — A2A's spec mentions a "directory of sub-agents." Herald could expose `https://pherald.example.com/.well-known/agent.json` as a directory referencing per-tenant sub-agents. Most idiomatic A2A but most complex.

Recommendation: option 2 for MVP; option 3 as Wave 4b+1.

§3.5 JWT auth integration

A2A v1.0 has Bearer / OAuth 2.0 / mTLS / Signed AgentCards. Herald's `commons_auth/` already does JWT verification; the natural integration is:

- AgentCard declares `authentication.schemes: ["Bearer"]`.
- Every `message/send` request carries `Authorization: Bearer <jwt>` where the JWT is issued by Herald's existing JWKS.
- `tenant_id` extracted from JWT claim.
- For Signed AgentCards (v1.0 feature): Herald signs its AgentCard with the same JWKS; consumers verify with the same JWKS endpoint.

§3.6 Idempotency

A2A's `message/send` has no native idempotency key. Herald enforces via:

- `metadata.idempotency_key` in the `params.message.metadata` — Herald's required extension.
- Same Redis SETNX path as REST.

Document the contract in the AgentCard's `skills[].description`.

§3.7 OTEL propagation

- HTTP transport — `traceparent + tracestate` headers propagate natively.
- gRPC transport — OTEL's standard gRPC propagator.
- The A2A spec doesn't define trace context propagation explicitly; Herald uses the HTTP header convention.

§3.8 Failure modes specific to Herald

1. **AgentCard discovery DoS.** Anyone can fetch `/well-known/agent.json`. Cache aggressively (Herald already has Brotli + ETag + 1-hour cache headers).
2. **Task long-poll resource exhaustion.** Streaming tasks tie up HTTP connections. Limit concurrent streaming tasks per tenant.
3. **Signed AgentCard rotation.** When Herald rotates its JWKS, the AgentCard's signature breaks. Auto-resign on key rotation; document the rotation cadence.

§4. Step-by-step implementation guide for Herald

§4.1 Add the Go SDK as a submodule

```
# from repo root — DO NOT actually run; plan only
git submodule add git@github.com:a2aproject/a2a-go.git
    submodules/go-a2a-sdk
git -C submodules/go-a2a-sdk checkout v2.3.1
```

Min Go version: 1.24.4+. Apache 2.0. 381 stars.

Alternative: spec V3 §9.1 `go get carve-out (go get github.com/a2aproject/a2a-go/v2@v2.3.1)`.

§4.2 New module commons_a2a/

commons_a2a/

```
├── go.mod
├── server/
│   ├── card.go           # AgentCard publisher (with JWS signing for v1)
│   ├── jsonrpc.go        # JSON-RPC binding
│   ├── rest.go           # REST binding
│   ├── grpc.go           # gRPC binding (optional)
│   ├── executor.go       # AgentExecutor interface wiring to Herald's d
│   ├── stream.go         # SSE streaming task updates
│   ├── push.go           # push notification webhook poster
│   ├── auth.go           # JWT/OAuth/mTLS layering
│   └── server_test.go
├── client/
│   ├── client.go         # outbound A2A client (Herald → other agent)
│   └── client_test.go
└── e2e_real_a2a_cli_test.go # real `a2a` CLI invocation
```

§4.3 Wire A2A into every flavor's serve command

Add to commons/cli/serve.go:

```
cmd.Flags().Bool("a2a", true, "publish A2A AgentCard at /.well-known/agent.json and serve at /a2a (default: on)")
cmd.Flags().Bool("a2a-grpc", false, "additionally serve A2A over gRPC at :7c-grpc")
```

Per-flavor wiring: each flavor's cmd/<flavor>/serve.go calls commons_a2a/server.Register(executor) where executor implements the flavor's task→business-logic mapping.

§4.4 AgentCard catalog per flavor

Flavor Skills

pherald notify, bulk_notify, subscribe, query_events

cherald query_compliance, attest, evidence_collect

sherald query_safety, acknowledge_safety_event

bherald query_budget

rherald query_risk, flag_risk

iherald acknowledge_incident, resolve_incident, query_incident

scherald enqueue_job, cancel_job, query_schedule

§4.5 New e2e_bluff_hunt invariants

- **E56** — curl /.well-known/agent.json on a running pherald; assert valid JSON + name field + skills array.
- **E57** — A2A message/send JSON-RPC POST; assert task created + status completed or working.
- **E58** — A2A message/stream SSE; assert N progress events.
- **E59** — A2A tasks/get; assert task lifecycle.

- **E60** — A2A push notification: `tasks/pushNotificationConfig/set` + long-running task → assert webhook POST hits the configured URL.
- **E61** — A2A Signed AgentCard JWS verification: fetch card, JWS verify against Herald JWKS, assert valid signature.
- **E62** — Cross-protocol consistency: same `notify` via MCP (Test 2 from MCP.md) and via A2A (E57) produces IDENTICAL row in `events_processed`.
- **E63** — JWT-less A2A request → 401 (auth proof).

§4.6 Mutation gates

- Mutation: remove the AgentCard route handler → E56 FAILS.
- Mutation: remove the JWT middleware on `/a2a` → E63 FAILS.
- Mutation: stub out the signing logic → E61 FAILS.
- Mutation: break the push-notification poster → E60 FAILS.

§4.7 Spec V3 amendments

Add to `docs/specs/mvp/specification.V3.md`:

- **§4y.0** — A2A AgentCard structure + URL convention.
- **§4y.1** — A2A skills catalog per flavor (mapping to existing Herald operations).
- **§4y.2** — auth via JWT-as-Bearer + Signed AgentCards.
- **§4y.3** — `tenant_id` propagation via JWT.
- **§4y.4** — `idempotency_key` as metadata extension.
- **§4y.5** — task lifecycle + push notification webhook semantics.

§4.8 HRD scaffolding

- **HRD-230** — bootstrap `commons_a2a/` module.
- **HRD-231** — implement AgentCard publisher.
- **HRD-232** — implement A2A JSON-RPC binding.
- **HRD-233** — implement A2A REST binding.
- **HRD-234** — implement A2A SSE streaming.
- **HRD-235** — implement push notifications.
- **HRD-236** — Signed AgentCards (v1.0).
- **HRD-237** — gRPC binding (opt-in).
- **HRD-238** — per-flavor skill mapping (×7 flavors).
- **HRD-239** — A2A client wrapper for outbound calls.
- **HRD-240** — JWT-as-A2A-Bearer.
- **HRD-241** — idempotency metadata extension.
- **HRD-242** — Cross-protocol consistency tests (MCP + A2A → same row).
- **HRD-243** — `e2e_bluff_hunt` E56–E63.
- **HRD-244** — mutation gate suite for A2A.
- **HRD-245** — spec V3 §4y amendments.
- **HRD-246** — operator credentials guide update.
- **HRD-247** — A2A registry publication (optional — list Herald in A2A registries).
- **HRD-248** — load testing (concurrent tasks).
- **HRD-249** — operator runbook for AgentCard rotation.
- **HRD-250** — close Wave 4b.

§5. §107 anti-bluff testing strategy

The bar: a developer running Google ADK / BeeAI / LangGraph with their own A2A-compatible agent can configure Herald as a remote agent, see Herald's AgentCard resolve, send notify tasks, and receive successful responses that match REST byte-for-byte (mod transport framing).

§5.1 Happy-path tests

- **Test 1: AgentCard discovery.** `curl https://pherald-test.local:7104/.well-known/agent.json` → assert HTTP 200, valid JSON, name, version, url, skills[] populated. **Physical proof: byte-diff against golden/agent_card.pherald.json.**
- **Test 2: message/send round-trip.** Send valid notify task → assert response includes task.id, task.status: completed, task.artifacts[0] containing the notification receipt. **Physical proof: same row in events_processed Postgres table.**
- **Test 3: message/stream SSE.** Send a long-running task → consume SSE stream → assert at least one TaskStatusUpdateEvent with status working before completed. **Physical proof: SSE event log captured + matches expected sequence.**
- **Test 4: push notification.** Register webhook URL; submit long-running task; assert HTTP POST hits the webhook within timeout. **Physical proof: webhook server (test harness) logged the POST.**

§5.2 Edge cases

- **Test 5: AgentCard caching.** Repeated `curl` should hit ETag 304. Assert If-None-Match short-circuit.
- **Test 6: malformed JSON-RPC.** Send invalid envelope → JSON-RPC -32600 / -32700.
- **Test 7: oversized message.** 10 MiB message → 413 or JSON-RPC -32600.
- **Test 8: streaming reconnect.** SSE disconnect mid-stream → tasks/resubscribe resumes from Last-Event-ID.

§5.3 Failure modes

- **Test 9: auth fail.** No Bearer → 401. Invalid signature → 401. Expired token → 401.
- **Test 10: Signed AgentCard tampering.** Modify a byte in AgentCard JWS → JWS verify FAILS → consumer-side test rejects card.
- **Test 11: cross-tenant.** Tenant A's JWT submits notify referencing Tenant B's subscriber → 403.
- **Test 12: task cancellation.** tasks/cancel on in-progress task → assert status transitions to canceled within 5s.

§5.4 Performance

- **Test 13: throughput.** 1000 concurrent message/send (sync, blocking) → p50 < 50ms, p99 < 500ms.
- **Test 14: streaming overhead.** SSE vs blocking — SSE adds < 10ms p50 overhead.

§5.5 Concurrency

- **Test 15: 100 concurrent streaming clients.** All complete; no deadlock.

§5.6 Real-client integration

scripts/e2e_a2a_real_cli.sh:

1. Start Herald flavor binary with `--a2a`.
2. Install a2a CLI: `go install github.com/a2aproject/a2a-go/v2/cmd/a2a@v2.3.1`.
3. Run `a2a resolve https://localhost:7104/.well-known/agent.json` → assert resolves.
4. Run `a2a send --skill notify --json '{...}'` → assert successful task.
5. Diff REST equivalent → assert `events_processed` row matches.

§5.7 Cross-protocol consistency

A `notify` invocation via REST, MCP, A2A MUST yield IDENTICAL rows in `events_processed` and IDENTICAL outbound channel hits. Test runs all three paths sequentially with deterministic payload; asserts exact equivalence.

§5.8 Mutation gates

Each happy-path test has a paired mutation:

- **Mutation A:** delete AgentCard handler → Test 1 FAILS.
- **Mutation B:** stub the task executor → Test 2 FAILS with no task created.
- **Mutation C:** disable SSE flush → Test 3 FAILS (stream silent).
- **Mutation D:** break the JWS signer → Test 10 PASSES with tampering not detected (worse: existing valid signature breaks).

§5.9 Wire-level inspection

- `tcpdump` on `:7104` during Test 4 → assert push POST body matches expected JSON shape.
- `openssl s_client / curl -v` → assert TLS handshake + cert.
- `jose-util / openssl rsautl` → verify the JWS-signed AgentCard offline.

§6. Open questions for operator

1. **Default transport — JSON-RPC vs REST vs gRPC?** Recommend JSON-RPC + REST as default (HTTP-native); gRPC opt-in.
2. **One AgentCard per tenant, or one AgentCard + JWT tenant?** Recommend single AgentCard + JWT.
3. **Signed AgentCards on day 1?** Recommend yes (v1.0 feature; cheap; major trust upgrade).
4. **List Herald in public A2A registries?** Marketing decision.
5. **Push notification retry policy?** Reuse Standard-Webhooks delivery from `commons_webhook/` (Wave 4d).
6. **A2A skills as "thin wrappers" over REST or first-class?** Recommend thin wrappers — DRY.
7. **gRPC binding worth it?** Defer; revisit if a major operator (e.g. Google ADK) demands it.
8. **AgentCard JWKS endpoint URL?** Reuse Herald's existing JWKS at `/.well-known/jwks.json`.

§7. References

(All fetched 2026-05-22.)

- **Announcement.** "Announcing the Agent2Agent Protocol (A2A)" — Google Developers Blog, 2025-04-09 — <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>
- **Spec repo.** <https://github.com/a2aproject/A2A> — 23.9k stars, v1.0.0 (2026-03-12), Linux Foundation governance.
- **Official Go SDK.** <https://github.com/a2aproject/a2a-go> — v2.3.1 (2026-05-13), 381 stars, Apache 2.0, Go 1.24.4+.
- **Spec website.** <https://google.github.io/A2A/>
- **ADK integration.** <https://google.github.io/adk-docs/a2a/>
- **A2A v1.0 announcement.** Spec page covering Signed AgentCards.
- **Alternative implementations.**
 - <https://github.com/yeeaiclub/a2a-go>
 - <https://github.com/trpc-group/trpc-a2a-go>
 - <https://github.com/TheApeMachine/a2a-go> — interesting because it documents MCP-A2A interop.
- **Stellagent A2A explainer.** <https://stellagent.ai/insights/a2a-protocol-google-agent-to-agent>
- **Survey arxiv:2505.02279** — MCP/ACP/A2A/ANP comparison.

§8. Catalogue-check verdict

Per §11.4.74:

- **vasic-digital:** no module matching `a2a` or `agent2agent`. No-match.
- **HelixDevelopment:** no module matching `a2a` or `agent2agent`. No-match.

Verdict: no-match → vendor as **Herald-internal submodule** under `submodules/go-a2a-sdk/` pointing at `github.com/a2aproject/a2a-go@v2.3.1`. Alternative: spec V3 §9.1 `go get` exception.